

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

rehmang110_petdataset_path = kagglehub.dataset_download('rehmang110/petdataset')
rehmang110_petdatasett_path = kagglehub.dataset_download('rehmang110/petdatasett')

print('Data source import complete.')
```

```
import numpy as np
import pandas as pd
import torch
import torchvision
from torchvision import models , transforms
import cv2
import matplotlib.pyplot as plt
from PIL import Image
import os
import shutil
from torch.utils.data import DataLoader , Dataset
from tqdm import tqdm
from pathlib import Path
import torch.nn as nn
import torch.optim as optim
from torchvision.models.segmentation import deeplabv3_resnet50
from tqdm import tqdm
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

[Show hidden output](#)

```
print("Versions:")
print(f"numpy: {np.__version__}")
print(f"pandas: {pd.__version__}")
print(f"torch: {torch.__version__}")
print(f"torchvision: {torchvision.__version__}")
print(f"cv2 (OpenCV): {cv2.__version__}")
print(f"PIL (Pillow): {Image.__version__}")
print(f"pathlib: built-in (no version)")
```

```
Versions:
numpy: 1.26.4
pandas: 2.2.3
torch: 2.6.0+cu124
torchvision: 0.21.0+cu124
cv2 (OpenCV): 4.11.0
PIL (Pillow): 11.2.1
pathlib: built-in (no version)
```

```
if torch.cuda.is_available():
    device = "cuda"
    dtype = torch.float16

else:
    device = "cpu"
    dtype = torch.float32

print(f"Device is :{device}")
print(f"dtype is :{dtype}")
```

```
Device is :cuda
dtype is :torch.float16
```

```
# Source and destination paths
src_path = Path("/kaggle/input/petdataset/annotations/annotations")
dest_path = Path("/kaggle/working/annotations")
dest_path.mkdir(parents=True, exist_ok=True)

for item in src_path.iterdir():
    if item.is_file():
        shutil.copy(item, dest_path / item.name)
        print(f"Copied file: {item.name}")

    elif item.is_dir():
        dest_dir = dest_path / item.name
        dest_dir.mkdir(parents=True, exist_ok=True)

        for file in item.iterdir():
            if file.is_file():
                shutil.copy(file, dest_dir / file.name)
```

[Show hidden output](#)

```
# Source and destination paths
src_path = Path("/kaggle/input/petdataset/images/images")
dest_path = Path("/kaggle/working/images")
dest_path.mkdir(parents=True, exist_ok=True)

for item in src_path.iterdir():
    if item.is_file():
        shutil.copy(item, dest_path / item.name)
        print(f"Copied file: {item.name}")

    elif item.is_dir():
        dest_dir = dest_path / item.name
        dest_dir.mkdir(parents=True, exist_ok=True)

        for file in item.iterdir():
            if file.is_file():
                shutil.copy(file, dest_dir / file.name)
```

[Show hidden output](#)

```
# here we downlaod the dataset but we have manually downlaod
#!wget https://www.robots.ox.ac.uk/~vgg/data/pets/data/images.tar.gz
#!wget https://www.robots.ox.ac.uk/~vgg/data/pets/data/annotations.tar.gz
#!tar -xvf images.tar.gz
#!tar -xvf annotations.tar.gz
```

```
--2025-07-30 12:07:16-- https://www.robots.ox.ac.uk/~vgg/data/pets/data/images.tar.gz
Resolving www.robots.ox.ac.uk (www.robots.ox.ac.uk)... 129.67.94.2
Connecting to www.robots.ox.ac.uk (www.robots.ox.ac.uk)|129.67.94.2|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://thor.robots.ox.ac.uk/pets/images.tar.gz [following]
--2025-07-30 12:07:16-- https://thor.robots.ox.ac.uk/pets/images.tar.gz
Resolving thor.robots.ox.ac.uk (thor.robots.ox.ac.uk)... 129.67.95.98
Connecting to thor.robots.ox.ac.uk (thor.robots.ox.ac.uk)|129.67.95.98|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 791918971 (755M) [application/octet-stream]
Saving to: 'images.tar.gz'
```

```
images.tar.gz      0%[                ] 6.68M 34.3KB/s   eta 6h 26m
```

```
# here we see our dataset how looks like
images_dir = Path("/kaggle/working/images")
images_list = [item.name for item in images_dir.iterdir()]
print(images_list[:10])
```

```
['Russian_Blue_34.jpg', 'newfoundland_103.jpg', 'shiba_inu_143.jpg', 'leonberger_170.jpg', 'Maine_Coon_115.jpg', 'yorkshire_101.jpg', 'border_collie_102.jpg', 'border_collie_103.jpg', 'border_collie_104.jpg', 'border_collie_105.jpg']
```

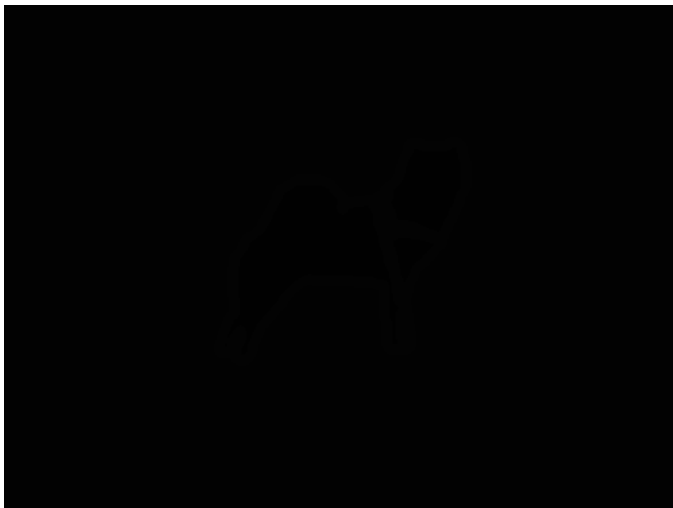
```
image_path = images_dir / "shiba_inu_112.jpg"
Image.open(image_path)
```



```
# here we see our dataset how looks like
masks_dir = Path("/kaggle/working/annotations/trimaps")
masks_list = [item.name for item in masks_dir.iterdir()]
print(masks_list[:10])
```

```
['._yorkshire_terrier_83.png', '._basset_hound_76.png', 'keeshond_120.png', '._boxer_27.png', 'newfoundland_59.png']
```

```
mask_path = masks_dir / "shiba_inu_112.png"
Image.open(mask_path)
# they all seem black, but it actually look to your eye, because 1,2,3 values are black we do not much differentiate
```



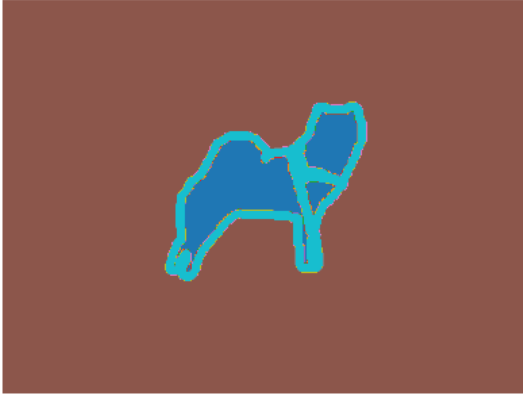
```
# now this help us to visualize the mask properly and you clearly see the image and its mask
```

```
# Load mask (as grayscale)
mask_path = mask_path
mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)

# Fix labels: original labels are 1, 2, 3 - we shift them to 0, 1, 2
mask = mask - 1 # Now: 0 = background, 1 = border, 2 = pet

# Display with color mapping
plt.figure(figsize=(5,5))
plt.imshow(mask, cmap="tab10") # apply a visible color map
plt.title("Segmentation Mask (colored)")
plt.axis('off')
plt.show()
```

Segmentation Mask (colored)



```

from torch.utils.data import Dataset
from pathlib import Path
from PIL import Image
import numpy as np
import torch

class PetDataset(Dataset):
    def __init__(self, image_dir, mask_dir, file_names, transform=None):
        self.image_dir = image_dir
        self.mask_dir = mask_dir
        self.file_names = file_names
        self.transform = transform

    def __len__(self):
        return len(self.file_names)

    def __getitem__(self, idx):
        # Get the filename (without extension)
        name = self.file_names[idx]

        # Load image and mask paths
        img_path = Path(self.image_dir) / f"{name}.jpg"
        mask_path = Path(self.mask_dir) / f"{name}.png"

        # Load the image and mask
        image = Image.open(img_path).convert("RGB")
        mask = Image.open(mask_path)

        # Resize both to same size
        image = image.resize((128, 128))
        mask = mask.resize((128, 128), resample=Image.NEAREST)

        # Convert mask to numpy and shift class values from 1,2,3 → 0,1,2
        mask = np.array(mask) - 1
        mask = torch.tensor(mask, dtype=torch.long)

        # Apply image transforms (mask is left as-is)
        if self.transform:
            image = self.transform(image)

        return image, mask

```

```

image_transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

```

```

print(images_dir)
print(masks_dir)

```

```
/kaggle/working/images
/kaggle/working/annotations/trimaps
```

```
file_names = [f.stem for f in images_dir.glob("*.jpg")]

dataset = PetDataset(images_dir, masks_dir, file_names, transform=image_transform)

# Check one sample
image, mask = dataset[0]

print("Image shape:", image.shape) # [3, H, W]
print("Mask shape:", mask.shape)   # [H, W]
print("Mask unique values:", torch.unique(mask))

Image shape: torch.Size([3, 128, 128])
Mask shape: torch.Size([128, 128])
Mask unique values: tensor([0, 1, 2])
```

```
train_loader = DataLoader(dataset, batch_size=2, shuffle=True, num_workers=2)
```

```
# now we view this train_loader
img_train, img_mask = next(iter(train_loader))
print(img_train.shape) # expect (8,3,256,256) batch_size include
print(img_mask.shape)  # expect (8, H, W)
print(len(train_loader)*8) # number of files
```

```
torch.Size([2, 3, 128, 128])
torch.Size([2, 128, 128])
29560
```

```
# load the pre Trained model
model = torchvision.models.segmentation.deeplabv3_resnet50(pretrained=False, num_classes=2)
# Change classifier head to match 3 classes (background, border, pet)
model.classifier[4] = torch.nn.Conv2d(256, 3, kernel_size=1)
model.to(device)
```

[Show hidden output](#)

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)
```

```
def train_epochs(num_epochs):
    model.train()

    for epoch in range(num_epochs):
        total_loss = 0

        for images, masks in train_loader:
            images = images.to(device)
            masks = masks.to(device)

            outputs = model(images)['out']
            loss = criterion(outputs, masks)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        avg_loss = total_loss / len(train_loader)
        print(f"Epoch {epoch+1}, Avg Loss: {avg_loss:.4f}")
```

```
train_epochs(num_epochs=1)
```

[Show hidden output](#)

